

Discourse-JEPA: Latent World Models over Reasoning Steps

Method and first experimental cycle — fully reproducible from these slides

TextJEPA project

July 9, 2026 (repo: `TextJEPA/`, runs: `runs/disc_*`)

Idea in one slide

- ▶ Treat a **reasoning trace** as trajectories of a world model: the *state* s_t is a compressed discourse state (“what has been established”), the *action* a_t is a tiny latent code of an *intent phrase* for the next step.
- ▶ Actions state intent only (“*derive purple apples from orange jars times green pens .*”) — never the outcome (“ $= 5$ ”). The predictor $F(s_t, a_t)$ must therefore **compute consequences in latent space**.
- ▶ Generation = **search over actions**: roll candidate actions forward with F , score with an energy, execute the best in the environment, re-encode, replan (closed-loop MPC). **No decoder, no token loss anywhere** (JEPA-pure; the symbolic environment renders chosen actions via templates).
- ▶ Synthetic reasoning world $\Rightarrow$ exact ground truth for **linear probes** and **objective planning success**.

World: iGSM-style synthetic reasoning (exact generator)

Problem sampler (rejection-sampled until $3 \leq |\mathcal{A}_q| \leq 9$; rng = Random(f"{seed}:{index}")):

- ▶ $n \sim \mathcal{U}\{6, \dots, 12\}$ variables; names = distinct (adjective, noun) pairs from 20×20 vocabulary.
- ▶ Variable i : with $p=0.35$ (always for $i < 2$) a **leaf** $x_i = c_i$, $c_i \sim \mathcal{U}\{0..22\}$; else an **internal** $x_i = x_a \circ x_b \bmod 23$, $\circ \in \{+, -, \times\}$, parents a, b sampled from $\{0..i-1\}$.
- ▶ Query q = uniform over internal variables; $\mathcal{A}_q$ = ancestors of q (incl. q) = the necessary computation; the rest are **distractors**.

Training traces: solve by repeatedly resolving a feasible variable (parents resolved); with $p=0.15$ (max 2) pick a feasible *distractor* instead of a necessary one $\Rightarrow$ off-path states for the value head. Fresh problems every epoch (train seed 1, val seed 2; 100k/5k).

A verbatim sample (val problem #3)

Prompt (definitions shuffled per-sample, question last; one sentence = one chunk):

the number of purple apples equals the number of orange jars times the number of green pens .

the number of orange jars is 15 . the number of green pens is 8 .

(+ 7 distractor definitions) how many purple apples are there ?

Solution trace — step sentence (observed outcome) vs. action intent phrase (no outcome):

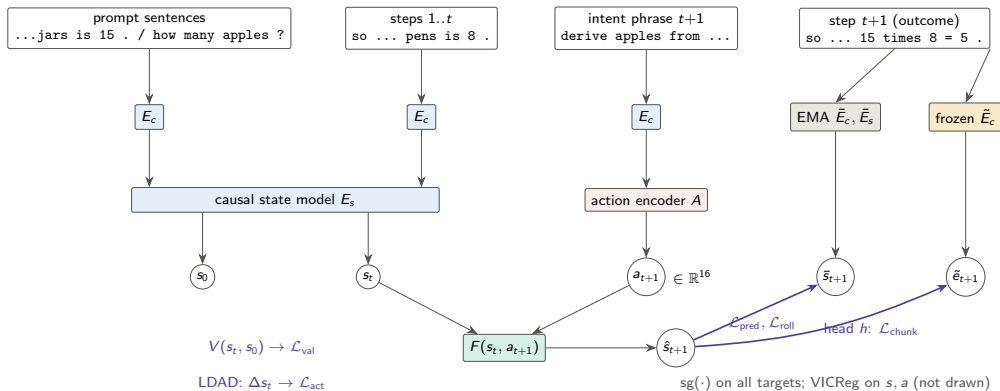
step sentence (encoder sees)	intent phrase (action encoder sees)
so the number of green pens is 8 .	look up the number of green pens .
so the number of orange jars is 15 .	look up the number of orange jars .
so the number of purple apples is	derive purple apples from orange
15 times 8 = 5 .	jars times green pens .

Note $15 \times 8 = 120 \equiv 5 \pmod{23}$: outcomes are nontrivial to predict.

Architecture (all sizes exact; 8.8–9.1M params)

- ▶ **Chunk encoder** E_c : word-level vocab (441 tokens), embed 256, 2-layer pre-norm transformer (4 heads, FF 1024), masked mean-pool $\rightarrow$ one 256-d vector per sentence. Shared for prompt, steps, intents.
- ▶ **State model** E_s : 4-layer *causal* transformer (8 heads, FF 1024) over [prompt chunks | step chunks] + learned positions + 2 segment embeddings. s_0 = hidden at the question; s_t = hidden at step t .
- ▶ **Action encoder** A : MLP $256 \rightarrow 256 \rightarrow 16$ on the intent-pharse embedding (optional FSQ, unused in main runs).
- ▶ **Predictor** F : residual MLP, $\hat{s}_{t+1} = s_t + \text{MLP}([\text{LN}(s_t); a_t])$, 2 hidden layers of 1024 (GELU).
- ▶ **Hierarchy**: CLS-transformer over $K=3$ action codes $\rightarrow$ 8-d macro action m_t ; $F_{\text{hi}}(s_t, m_t)$ predicts $\bar{s}_{t+3}$ (same latent space, HWM-style).
- ▶ **LDAD** (Delta-JEPA): MLP on $\Delta s = s_{t+1} - s_t \rightarrow$ op logits (4) + regression to the EMA intent embedding.
- ▶ **Value head** V : MLP on $[\text{LN}([s_t; s_0])]$ $\rightarrow$ scalar = predicted remaining necessary steps.
- ▶ **Teachers**: EMA copies of E_c, E_s (momentum $0.99 \rightarrow 0.999$ linear); **frozen anchor** $\tilde{E}_c$ = copy of E_c at random init, never updated.

Training pipeline



Objectives (exact losses and weights)

Let $d(x, y) = \text{smoothL1}(\text{LN}(x), \text{LN}(y))$ (per-dim mean; LN = parameter-free layer norm), masks over valid steps.

$$\mathcal{L}_{\text{pred}} = d(F(s_t, a_t), \text{sg } \bar{s}_{t+1}) \quad \text{weight 1.0}$$

$$\mathcal{L}_{\text{roll}} = d(F^{(t)}(s_0, a_{1:t}), \text{sg } \bar{s}_t) \text{ (open loop, all } t) \quad 1.0$$

$$\mathcal{L}_{\text{hi}} = d(F_{\text{hi}}(s_t, m_t), \text{sg } \bar{s}_{t+3}) \quad 0.5$$

$$\mathcal{L}_{\text{vic}} = \sum_j \max(0, 1 - \text{std}(s^{(j)})) + 0.04 \|\text{offdiag}(\text{Cov}(s))\|_F^2 / D + 0.1 \text{var}(a) \quad 1.0$$

$$\mathcal{L}_{\text{act}} = \text{CE}(\text{op} \mid \Delta s_t) + d(g(\Delta s_t), \text{sg } \tilde{E}_c(u_t)) \text{ (Delta-JEPA)} \quad 2.0$$

$$\mathcal{L}_{\text{val}} = \text{smoothL1}(V(s_t, s_0), \# \text{unresolved necessary}) \quad 1.0$$

$$\mathcal{L}_{\text{chunk}} = d(h(\hat{s}_{t+1}), \text{sg } \tilde{E}_c(y_{t+1})) + 0.5 d(h(\text{rollout}_t), \text{sg } \tilde{E}_c(y_t)) \quad 0 / 2.0$$

$\mathcal{L}_{\text{chunk}}$: weight 0 in `disc_base`; 2.0 in `disc_chunkpred/combo` with the **frozen** anchor $\tilde{E}_c$.
Value head input detached except in `disc_valgrad/combo`.

Training protocol

- ▶ AdamW, lr 3×10^{-4} , weight decay 0.05 (none for dim < 2 params), $\beta = (0.9, 0.95)$; cosine schedule to floor 0.05, 1000-step warmup; grad clip 5.0.
- ▶ Batch 128 problems; 20 epochs $\times$ 781 steps; **fresh data every epoch** (no memorization confound). EMA momentum $0.99 \rightarrow 0.999$ linear over training.
- ▶ Hardware: one V100-32GB, fp32, ~ 40 min/run.
- ▶ In-training eval each epoch: all losses + collapse diagnostics (per-dim std, RankMe effective rank) + closed-form ridge probes.
- ▶ Code: `scripts/train.py +experiment=...`; every run's exact config is stored in its checkpoint.

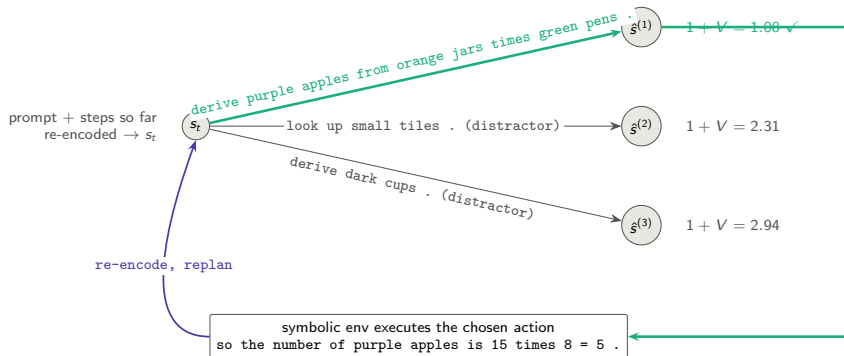
Planning: closed-loop MPC by exhaustive latent search

Interface knowledge (legitimate): feasible actions = unresolved variables whose stated dependencies are resolved — readable from the prompt. **Consequence knowledge**: latent only.

1. Encode prompt + steps so far $\rightarrow s$ (and s_0 once).
2. Enumerate feasible action *sequences* to depth L (lookahead; cap 64), encode their intent phrases $\rightarrow a_i$.
3. Roll out $\hat{s} \leftarrow F(\hat{s}, a_i)$ along each sequence.
4. Score: cost = steps taken + $V(\hat{s}_{\text{end}}, s_0)$ (estimated total steps).
5. Execute the first action of the argmin in the symbolic environment $\rightarrow$ real outcome sentence appended; go to 1.

Success = query resolved within the **strict optimal budget** $|\mathcal{A}_q|$ (slack 0: any distractor pick is fatal). 200 val episodes. Discrete enumeration replaces CEM; a sampled action prior would need CEM/MPPI.

Planning: search graph (one replanning round, lookahead 1)



V values are predicted *remaining necessary steps* of the predicted next latent; lookahead 2 scores 2-step latent rollouts.

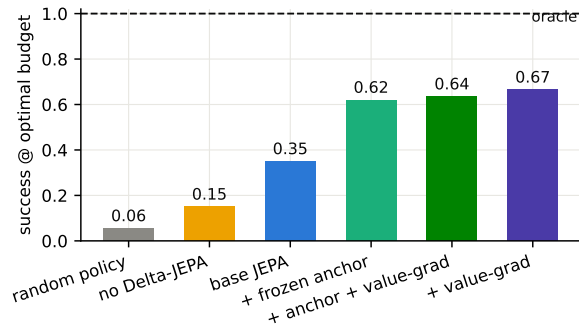
Experiment grid (each = one training run, 20 epochs)

run	$\mathcal{L}_{\text{chunk}}$	anchor	value grad $\rightarrow$ encoder	other
disc_base	—		no	reference
disc_no_delta	—		no	$\mathcal{L}_{\text{act}}$ weight 0
disc_chunkpred	frozen, w=2.0		no	—
disc_valgrad	—		yes	—
disc_combo	frozen, w=2.0		yes	—
disc_value_only	—		yes	<i>all</i> JEPA weights 0
disc_frozen_enc	—		no	encoders frozen at init

Baselines at plan time: **random feasible policy**, **symbolic oracle** (=100%), and **oracle goal-distance energy** (LeWM-style, uses the encoded solved state — diagnostic only).

Result 1 — training objectives determine plannability

Claim: JEPA losses turn a frozen-vocabulary encoder into a plannable world model; each added mechanism helps.



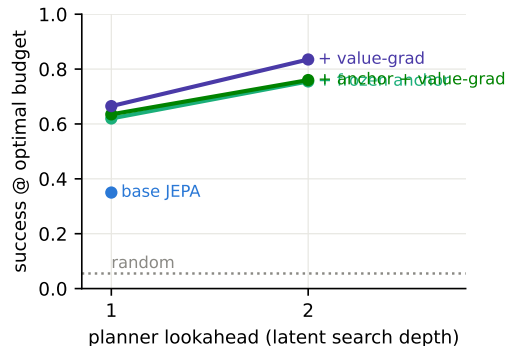
Success within the *strict optimal budget*, greedy lookahead 1, 200 episodes, mean necessary steps 4.07.

With slack 2: base 0.75, +anchor 0.88, +value-grad **0.92** (random 0.41).

Distractor pick rate: random 0.43 → base 0.28 → valgrad 0.14.

Result 2 — deeper latent search helps monotonically

Claim: Success grows with search depth: rollouts of F and the energy stay trustworthy over multiple imagined steps.



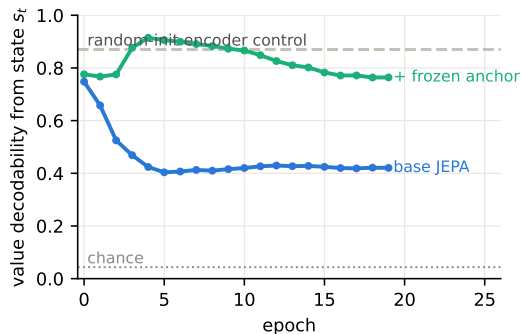
Lookahead 2 = exhaustive 2-step feasible sequences (cap 64), scored at the imagined end state.

disc_valgrad: 0.665 $\rightarrow$ **0.835**; distractor rate 0.14 $\rightarrow$ 0.07.

This is the planning-time compute knob promised by latent world models — no retraining involved.

Result 3 — encoder–predictor collusion, measured

Claim: Pure state-prediction JEPA *removes* outcome information that was linearly present at initialization; a frozen outcome anchor prevents it.



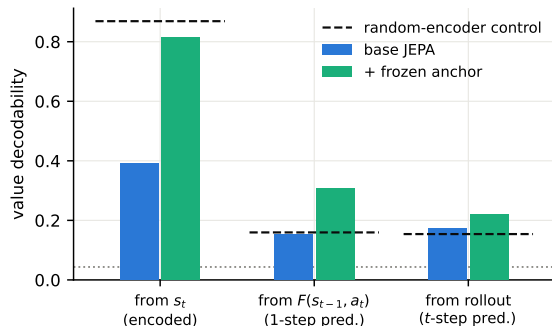
Probe: linear decodability of the just-computed value (23 classes) from s_t , val split.

Both loss sides are learned; EMA targets *trail the online encoder*, so deleting a hard-to-predict feature lowers the loss — stop-grad does not pin content.

Consequence in `disc_base`: answer decodable from the terminal state at only 0.26 (random-init control: 0.65).

Result 4 — latent arithmetic under the frozen anchor

Claim: With $\mathcal{L}_{\text{chunk}}$, the predictor computes outcomes it never observes: value decodability from *predicted* latents doubles the random-encoder control.



Dashed = random-init encoder control (surface memory, no computation); dotted = chance $1/23$.

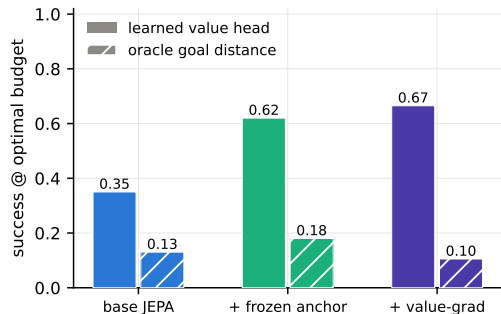
From $F(s_{t-1}, a_t)$: base 0.15 (= control) $\rightarrow$ anchor **0.31**.

Answer from terminal state: 0.26 $\rightarrow$ **0.98**.

Full-trace open-loop rollout: 0.22 (error compounds; matches Result 2's need for re-encoding).

Result 5 — the goal energy must be learned (LeWM test)

Claim: Raw latent distance to the oracle goal state plans barely above random; the trained value head is the energy that works.



Oracle goal = encoded terminal state of the *true* solution (planner gets it for free — upper-bound diagnostic).

Scoring $\|\text{LN}(\hat{s}) - \text{LN}(s_{\text{goal}})\|_1$: base 0.13, anchor 0.18, valgrad 0.105 (distractor rates $\approx$ random).

$\Rightarrow$ JEPA geometry alone is not a distance-to-goal metric here; distillation should target the *value head* (or an advantage head $A(s, a)$), not raw geometry.

Result 6 — Delta-JEPA ablation; stability

Claim: Displacement-level action decoding is load-bearing for planning (replicates Delta-JEPA, arXiv:2606.31232, in language).

	success@opt	success@slack2	distractor rate	op from Δs
disc_base	0.35	0.75	0.28	1.00
disc_no_delta	0.15	0.51	0.40	0.98*

*op stays decodable (it correlates with surface features) but transitions lose the *action-discriminative* structure the planner needs.

Stability (all runs): per-dim state std ≈ 1.03 , RankMe effective rank 220–243 / 256, no collapse; VICReg + EMA sufficed, discrete codes (FSQ) not needed.

Probe suite (champion vs. reference; random-encoder control)

probe (linear, val split)	base	chunkpred	random-enc
value of current step from s_t	0.39	0.81	0.87
value from $F(s_{t-1}, a_t)$ (latent arithmetic)	0.15	0.31	0.15
value from open-loop rollout	0.17	0.22	0.15
op type from Δs	1.00	1.00	0.97
step goal-relevance from Δs	0.89	0.93	0.89
remaining steps from s_t	0.44	0.61	0.41
resolved count from s_t	0.97	0.92	0.95
final answer from terminal state	0.26	0.98	0.62
answer from prompt alone s_0 (should be hard)	0.035	0.05	0.04
value-head MAE (remaining steps)	0.76	0.57	—

`disc_combo` adds value-grad: MAE **0.38** with the same collusion-free probes (state value 0.89).

Honest limitations & what this is not

- ▶ **iGSM-style, not iGSM**: flat DAG, binary mod-23 ops, one question template, templated English — no hierarchical categories / instance-vs-abstract parameters of the original benchmark.
- ▶ **Actions are observed intents**, not variational latents: the symbolic world offers an enumerable interface. The variational prior $p(a \mid s, g)$ (for CEM/MPPI over sampled actions) is future work.
- ▶ **The goal energy is supervised** (remaining-steps labels from the symbolic executor), i.e. learned oracle distance — Result 5 shows raw geometry does not replace it.
- ▶ Value labels shape only the value head unless valgrad; the op-CE in LDAD uses the executed action's type (JEPA-legit: actions are observed at training time).
- ▶ Text “generation” is action selection + template rendering; a learned renderer is orthogonal to the world model.

Next steps

- ▶ **Baselines in flight:** `disc_value_only` (no JEPA losses) and `disc_frozen_enc` (random frozen features) — do JEPA representations matter beyond the value head?
- ▶ **Advantage head** $A(s, a)$ trained on $\Delta(\text{remaining})$ and action *ranking* losses — sharper than $V(F(s, a))$, which conflates predictor and value error.
- ▶ **Hierarchical planning:** use the trained F_{hi} + macro codes to propose 3-step jumps, refine with F (HWM).
- ▶ **Unobserved actions:** variational $q(a \mid s_{t-1}, s_t)$ with KL to prior $p(a \mid s, g)$; inspect emergent action clusters.
- ▶ **Harder worlds:** faithful iGSM (hierarchy + category sums), larger moduli, paraphrase templates, natural-language noise; scale.